

Thesis Extract: Proof-theoretic Completeness argument for ill-founded proof system of μ -LTL

Nicolas Alfred Raymond Mousset

April 25, 2026

Abstract

In this extract of my thesis project, I show soundness and completeness of an ill-founded proof system $K_{\mu\omega}$ for μ -LTL. This is an intermediate result towards soundness and completeness of the finite proof system K_μ for μ -LTL which will be achieved by a reduction via a cyclic proof system.

1 Introduction

[Introduction WIP]

2 Syntax and Semantics of μ -LTL

Adopting the preliminaries from [2]:

Let $Var = \{x, y, z, \dots\}$ be a countable set of variables. We also define the set $Var^\perp := \{\neg x \mid x \in Var\}$ of negated variables, disjoint from Var . We call elements of $Lit = Var \cup Var^\perp$ literals, from which we define the formulas of μ -LTL via the following grammar:

$$\alpha ::= x \mid \neg x \mid \bigwedge \Gamma \mid \bigvee \Gamma \mid \bigcirc \alpha \mid \mu \varphi \mid \nu \varphi$$

With $x \in Var$, and Γ a finite set of formulas. We use σ as a meta-variable for μ or ν . Furthermore, we define the predicate:

$$\varphi ::= \lambda x. \alpha \text{ provided } \varphi \text{ is } x\text{-positive}$$

The $\bigcirc$ is the Next operator from the usual grammar of LTL, and the Until operator is captured by the following fixpoint formula:

$$\alpha U \beta \equiv \mu(\lambda x. \bigvee \{\alpha, \beta \wedge \bigcirc x\})$$

The formulas yielded by this grammar are in positive normal form, which means that negation only occurs at the level of atomic variables. For example, $\neg \bigwedge \Gamma$ is not in positive normal form, but $\bigwedge \{\neg x, y, \neg z, \dots\}$ is. We call a formula *V-positive* if $\neg x$ does not occur in the formula for every variable $x \in V \subseteq Var$. Additionally, we call a formula *x-positive* if $V = \{x\}$.

We follow the standard nomenclature from [1]: We use the usual notion of free and bound variable occurrences. Furthermore, we will refer to $\bigwedge$ and $\bigvee$ as *connectives*, $\bigcirc$ as *modal operator*, and μ and ν as *quantifiers*. Formulas of the type $\bigwedge \Gamma$ are called *conjunctions*, and $\bigvee \Gamma$ are called *disjunctions*, of the elements of Γ , which are called *disjuncts*. Formulas of the form $\sigma x. \varphi$ are called *quantified*. In the following, a the set of formulas is denoted $\mathcal{L}_\mu$, and a set L of Literals is *consistent* iff $x \notin L$ or $\neg x \notin L$ for each $x \in Var$.

We employ the same naming conventions as in [1]. Symbols $\alpha, \beta, \dots$ range over formulas, $\varphi, \psi, \chi, \dots$ range over predicates, and $\Gamma, \Delta, \dots$ range over finite sets of formulas. We use the following abbreviations:

$$\begin{array}{llll} \alpha \wedge \beta := \bigwedge \{\alpha, \beta\} & \bigcirc \Gamma := \{\bigcirc \alpha \mid \alpha \in \Gamma\} & \mu x. \alpha := \mu(\lambda x. \alpha) & \top := \bigwedge \emptyset \\ \alpha \vee \beta := \bigvee \{\alpha, \beta\} & & \nu x. \alpha := \nu(\lambda x. \alpha) & \perp := \bigvee \emptyset \end{array}$$

Furthermore, every predicate ϕ is required to be *non-vacuous*: x occurs free in it at least once. This ensures that fixpoint unfolding strictly increases complexity: $|\phi(\sigma\phi)| > |\sigma\phi|$ for every quantified formula $\sigma\phi \in \mathcal{L}_\mu$.

We define the *complexity* of a formula α , written $|\alpha|$, as follows:

$$\begin{aligned} |x| = |\neg x| &= 0 & |\bigcirc \alpha| &= |\alpha| + 1 & |\bigwedge \Gamma| &= \max\{|\alpha| + 1 \mid \alpha \in \Gamma\} \\ |\lambda x. \alpha| &= |\alpha| & |\sigma \varphi| &= |\varphi| + 1 & |\bigvee \Gamma| &= \max\{|\alpha| + 1 \mid \alpha \in \Gamma\} \end{aligned}$$

We continue the set-up from [1], and simulate negation of formulas and predicates as follows:

$$\begin{aligned} x^\perp &= \neg x & (\bigwedge \Gamma)^\perp &= \bigvee (\Gamma)^\perp & \Gamma^\perp &= \{\alpha^\perp \mid \alpha \in \Gamma\} & (\mu\varphi)^\perp &= \mu^\perp \varphi^\perp = \nu\varphi^\perp \\ (\neg x)^\perp &= x & (\bigvee \Gamma)^\perp &= \bigwedge (\Gamma)^\perp & (\bigcirc \alpha)^\perp &= \bigcirc \alpha^\perp & (\nu\varphi)^\perp &= \nu^\perp \varphi^\perp = \mu\varphi^\perp \\ (\lambda x. \alpha)^\perp &= \lambda x. \alpha[x \mapsto \neg x]^\perp \end{aligned}$$

Observe that $\alpha[x \mapsto \neg x]$ is the result of substituting $\neg x$ for all free occurrences of x in α , and that $\alpha[x \mapsto \neg x]^\perp$ is x -positive iff α is x -positive.

Exactly like in [1], the substitution can be generalized: We call a function $\theta : V \mapsto \mathcal{L}_\mu$ where $V \subseteq \text{Var}$ a *substitution*. We define $\eta[\theta]$, where η is a formula or predicate, as the result of substituting θx for x in η with a proper renaming of bound variables in η . This operation is commutative with connectives and quantifiers and is otherwise given by the following definition:

$$x[\theta] = \begin{cases} \theta x, & x \in \text{dom } \theta, \\ x & \text{otherwise,} \end{cases} \quad \neg x[\theta] = (\theta x)^\perp$$

$$(\lambda y. \alpha)[\theta] = \lambda y. \alpha[\theta], \text{ for } y \notin \text{dom } \theta \text{ and } y \text{ not free in the range of } \theta.$$

As stated in [1], we obtain:

Lemma 2.1. For any predicate or formula η , its complexity $|\eta| \leq |\eta[\theta]| \leq |\eta| + \max\{|\theta[x]| \mid s.t. x \in \text{dom } \theta\}$.

2.1 Subformulas and threads

Just like in [1], we fix two notions of subformula in μ -LTL. First, we call β a *literal subformula* of α iff β is among the formulas occurring in the recursive construction of α according to the grammar of $\mathcal{L}_\mu$ defined in section 2. This is the usual notion of subformula. Second, we call β a *derived subformula* of α iff β can be derived from α from the quantifier inference rules, this specifies $\varphi(\sigma\varphi)$ as a subformula of $\sigma\varphi$. Note that the complexity of a formula bounds the complexity of its literal subformulas, but not the complexity of its derived subformulas. In the following, the derived subformula plays a more important role than the literal subformula and we will often refer to the derived subformula as subformula. When we refer to the literal subformula, that will be made clear.

Definition 2.2. Subformula Graph

We call a *formula graph* a directed graph $\langle \mathcal{L}_\mu, \triangleright \rangle$ where $\triangleright$ is the *subformula relation* between formulas defined as follows: $\alpha \triangleright \beta$ iff either of the following three conditions hold:

$$(i) \alpha = \bigwedge \Gamma \text{ or } \alpha = \bigvee \Gamma, \text{ and } \beta \in \Gamma \quad (ii) \alpha = \bigcirc \beta \quad (iii) \alpha = \sigma\varphi \text{ and } \beta = \varphi(\alpha)$$

In the case when $\alpha \triangleright \beta$, we call β the *immediate subformula* of α . We also fix the reflexive transitive closure of $\triangleright$, denoted $\triangleright^*$, and called the *derived subformula relation*. A *derived subformula* of α is any β such that $\alpha \triangleright^* \beta$. We denote the set of all (derived) subformulas of a formula α as $\text{Sub}(\alpha)$.

Proposition 2.3. For every predicate φ not equal to $\lambda x. x$, $|\varphi(\sigma\varphi)| > |\sigma\varphi|$.

Remark 2.4. Every step $\alpha \triangleright \beta$ either strictly decreases complexity (cases (i) and (ii)) or strictly increases it (case (iii), by Lemma 2.3). No step preserves complexity. The degenerate case $\sigma(\lambda x. x) \triangleright \sigma(\lambda x. x)$ is a cyclic thread with no modal step, excluded by guardedness.

Definition 2.5. Threads

We call sequences $(\alpha_i)_{i < k}$ of formulas such that $\alpha_i \triangleright \alpha_{i+1}$ for each i *threads*. A sequence following the reflexive closure of $\triangleright$ is called a *weak thread*. A thread that starts at a formula α is called an α -thread. We

write $\tau : \alpha \triangleright^* \beta$ to express that τ is a finite α -thread ending in β , and we denote the length of the thread, as $|\tau|$, and $\tau(i)$ denotes the formula α_i which occurs at step i of the thread. That said, $\tau : \alpha \triangleright^* \beta$ expresses that $|\tau| > 0$, $\tau(0) = \alpha$, and $\tau(|\tau| - 1) = \beta$. In the case of $\beta = \alpha$, and $|\tau| > 1$, the thread τ is called a *cyclic thread*. Furthermore, writing $\tau : \alpha \triangleright_\eta^* \beta$ expresses that η is the formula in the thread with the lowest complexity of all the formulas in the thread. So there exists $i < |\tau|$ such that $\tau(i) = \eta$ and $|\eta| \leq |\tau(i)|$ for each $i \leq |\tau|$. Observe that if τ is a thread, then so is $\tau^\perp = (\tau(i)^\perp)_{i < |\tau|}$.

Definition 2.6. We call an infinite thread an ν -thread if the unique formula η as specified by the previous proposition has the form $\nu\varphi$. We call the thread an μ -thread otherwise, when η is of the form $\mu\varphi$.

Proposition 2.7. Substitution

Let θ be a substitution. If $\tau : \alpha[\theta] \triangleright^* \beta$, then either:

1. $\tau = (\tau'[\theta])\tau''$ for threads $\tau' : \alpha \triangleright^* x$ with $x \in \text{dom}(\theta)$, and $\tau'' : \theta(x) \triangleright^* \beta$.
2. $\tau = \tau'[\theta]$ for some thread $\tau' : \alpha \triangleright^* \beta$ where $\beta = \beta'[\theta]$.

Proposition 2.8. Minimal thread formula

Let τ be an infinite thread in $\text{Sub}(\alpha)$ for a (guarded) formula α . There is a unique formula η satisfying:

- (i) $\eta = \tau(i)$ for infinitely many i , and
- (ii) $|\eta| \leq |\tau(j)|$ for all but finitely many j .

Moreover, $\eta = \sigma\varphi$ for some $\sigma \in \{\mu, \nu\}$.

The proof of uniqueness requires an analysis of which formulas at the critical complexity n are reachable in the derived subformula closure. We isolate two auxiliary lemmas.

Lemma 2.9. Contexts increase complexity

Let $\sigma\varphi$ be a fixed-point formula with $\varphi = \lambda x.\alpha_0$. Then:

- (a) For any non-trivial context C and any formula β , $|C[\beta]| > |\beta|$.
- (b) Every literal subformula of $\alpha_0[\sigma\varphi/x]$ at complexity exactly $|\sigma\varphi|$ is an occurrence of $\sigma\varphi$ itself.

Proof. (a) By induction on the context C . The trivial context $C = [\cdot]$ is excluded by hypothesis. Otherwise the outermost constructor of C is a connective, $\bigcirc$, or a fixpoint binder. In each case the complexity definition gives $|C[\beta]| \geq |C'[\beta]| + 1$, where C' is the immediate sub-context containing the hole. If $C' = [\cdot]$, then $|C'[\beta]| = |\beta|$; otherwise C' is non-trivial and $|C'[\beta]| > |\beta|$ by the induction hypothesis. In either case, $|C[\beta]| > |\beta|$.

(b) Every literal subformula of α_0 (including α_0 itself) has complexity at most $|\alpha_0| = |\sigma\varphi| - 1 < |\sigma\varphi|$. The literal subformulas of $\alpha_0[\sigma\varphi/x]$ fall into four kinds:

1. x -free literal subformulas of α_0 : substitution acts as the identity on these, so they are literal subformulas of α_0 and have complexity at most $|\alpha_0| < |\sigma\varphi|$.
2. Substituted copies of $\sigma\varphi$: these have complexity exactly $|\sigma\varphi|$.
3. Proper literal subformulas of a substituted copy of $\sigma\varphi$: these have complexity strictly less than $|\sigma\varphi|$.
4. Non-trivial wrappers $C[\sigma\varphi]$ with $C \neq [\cdot]$, where C is a context arising from α_0 with x replaced by $\sigma\varphi$: by part (a), $|C[\sigma\varphi]| > |\sigma\varphi|$.

Only kind 2 contributes at complexity exactly $|\sigma\varphi|$. □

Lemma 2.10. Formulas at complexity n in the $\geq n$ subgraph

Let $\eta = \sigma\varphi \in \text{Sub}(\alpha)$ with $|\eta| = n$, and define

$$R_{\geq n}(\eta) = \{\beta \in \text{Sub}(\alpha) \mid \eta \triangleright^* \beta \text{ via a chain staying at complexity } \geq n\}.$$

Then every formula in $R_{\geq n}(\eta)$ at complexity exactly n is equal to η .

Proof. For $\beta \in \text{Sub}(\alpha)$, write $\text{syn}_n(\beta) = \{\gamma : \gamma \text{ is a literal subformula of } \beta, |\gamma| = n\}$. We prove by induction on the construction of $R_{\geq n}(\eta)$ that $\text{syn}_n(\beta) \subseteq \{\eta\}$ for every $\beta \in R_{\geq n}(\eta)$.

Since $R_{\geq n}(\eta) \subseteq \text{Sub}(\alpha)$ is finite, define $R_{\geq n}(\eta)$ as the closure: $R_0 = \{\eta\}$, $R_{i+1} = R_i \cup \{\gamma : \beta \in R_i, \beta \triangleright \gamma, |\gamma| \geq n\}$, stabilizing after finitely many steps. The induction is on the stage i at which β enters.

- *Base case:* $\beta = \eta$.
Then $\text{syn}_n(\eta) = \{\eta\}$: the formula $\eta = \sigma\varphi = \sigma(\lambda x. \alpha_0)$ has $|\alpha_0| < |\sigma\varphi| = n$, so every proper literal subformula of η has complexity $< n$.
- *Inductive step:* $\beta \in R_i$ with $\text{syn}_n(\beta) \subseteq \{\eta\}$, and $\beta \triangleright \gamma$ with $|\gamma| \geq n$.
We show $\text{syn}_n(\gamma) \subseteq \{\eta\}$.

Case (i): γ is a literal subformula of β (propositional or modal step). Then $\text{syn}_n(\gamma) \subseteq \text{syn}_n(\beta) \subseteq \{\eta\}$.

Case (ii): $\beta = \sigma'\psi$ is a fixed-point formula and $\gamma = \psi(\sigma'\psi)$. Write $\psi = \lambda x'. \alpha'_0$, so $\gamma = \alpha'_0[\sigma'\psi/x']$. We analyse $\text{syn}_n(\gamma)$ using Lemma 2.9 (with $\beta = \sigma'\psi$ playing the role of the substituted formula):

1. *Context parts* (literal subformulas of α'_0 not involving x'): these are literal subformulas of α'_0 , hence of $\sigma'\psi = \beta$. Those at complexity n are in $\text{syn}_n(\beta) \subseteq \{\eta\}$.
2. *The substituted copies* $\sigma'\psi = \beta$: if $|\beta| = n$, then β is a fixpoint formula at complexity n , and $\beta \in \text{syn}_n(\beta) \subseteq \{\eta\}$, so $\beta = \eta$. If $|\beta| > n$, then $|\beta| \neq n$, contributing nothing to $\text{syn}_n(\gamma)$.
3. *Proper literal subformulas of the substituted $\sigma'\psi$* : those at complexity n are in $\text{syn}_n(\sigma'\psi) = \text{syn}_n(\beta) \subseteq \{\eta\}$.
4. *Non-trivial wrappers* $C[\sigma'\psi]$ with $C \neq [\cdot]$: by Lemma 2.9, $|C[\sigma'\psi]| > |\sigma'\psi| = |\beta| \geq n$. So $|C[\sigma'\psi]| > n$, and such formulas do not contribute to $\text{syn}_n(\gamma)$.

Combining all cases: $\text{syn}_n(\gamma) \subseteq \{\eta\}$.

Now suppose $\beta \in R_{\geq n}(\eta)$ with $|\beta| = n$. Since $|\beta| = n$ and $\beta \in \text{syn}_n(\beta) \subseteq \{\eta\}$ (just proved), we conclude $\beta = \eta$. $\square$

Proof. Proof of Proposition 2.8

Existence

By Remark 2.4, complexity can decrease only finitely many times (it is bounded below by 0), so $(|\tau(i)|)_{i \geq 0}$ is eventually non-decreasing. Since $\text{Sub}(\alpha)$ is finite, some formula is visited infinitely often. Let $n = \min\{|\beta| \mid \beta \text{ visited infinitely often}\}$.

Some formula of complexity n visited infinitely often is a fixpoint formula

Suppose, for contradiction, that *no* formula of complexity n visited infinitely often is a fixpoint formula. Let η be any formula of complexity n visited infinitely often. Since η is not a fixpoint formula, every departure from η is via (i) or (ii), strictly decreasing complexity below n .

To return to complexity $\geq n$ (which must happen since η is visited infinitely often), the thread must pass through at least one fixpoint unfolding $\sigma\psi \triangleright \psi(\sigma\psi)$ with $|\sigma\psi| < n$ and $|\psi(\sigma\psi)| \geq n$. Since $\text{Sub}(\alpha)$ is finite, some such $\sigma\psi$ with $|\sigma\psi| < n$ is visited infinitely often, and hence so is $\psi(\sigma\psi)$. We consider three cases for $|\psi(\sigma\psi)|$:

- $|\psi(\sigma\psi)| > n$: then $\psi(\sigma\psi)$ is visited infinitely often at complexity $> n$, but all its immediate subformulas (obtained by propositional or modal decomposition) have complexity $< |\psi(\sigma\psi)|$. Since $\psi(\sigma\psi)$ is visited infinitely often, some formula reached from it is visited infinitely often at complexity $\leq n$. If at complexity $< n$, this contradicts minimality of n . If at complexity n , we are back to the situation of η . The thread must eventually return to $\psi(\sigma\psi)$ through another fixpoint unfolding from below n , but this yields no formula of complexity n that is a fixpoint, producing an infinite descent below n —contradicting minimality.

- $|\psi(\sigma\psi)| = n$: then $\psi(\sigma\psi)$ is visited infinitely often at complexity n . Since $\psi(\sigma\psi)$ contains $\sigma\psi$ as a proper sub-expression (with $|\sigma\psi| < n$), its outermost connective is propositional, modal, or a fixpoint binder. If a fixpoint binder, then $\psi(\sigma\psi)$ is a fixpoint formula of complexity n visited infinitely often, contradicting our assumption. If propositional or modal, every immediate subformula has complexity $< n$, but to reach $\psi(\sigma\psi)$ again the thread must pass through complexity $< n$, contradicting minimality of n (since some formula below n would be visited infinitely often).
- $|\psi(\sigma\psi)| < n$: contradicts minimality of n , since $\psi(\sigma\psi)$ is visited infinitely often.

In all cases we reach a contradiction. Therefore some formula of complexity n visited infinitely often is a fixpoint formula; call it $\eta = \sigma\varphi$.

Uniqueness

We first observe that the thread eventually stays at complexity $\geq n$: since $Sub(\alpha)$ is finite and each formula of complexity $< n$ is visited only finitely often (minimality of n), the total number of visits to complexity $< n$ is finite. In the eventually $\geq n$ regime, every visit to complexity exactly n is at a fixpoint formula: a non-fixpoint formula at complexity n can only be departed via a complexity-decreasing step to $< n$, which is impossible.

Now suppose $\eta' \neq \eta$ is another fixpoint formula of complexity n visited infinitely often. In the eventually $\geq n$ regime, the thread visits η and stays in $R_{\geq n}(\eta)$ (the set of formulas reachable from η by $\triangleright$ -steps staying $\geq n$) until it next returns to complexity n . By Lemma 2.10, the only formula at complexity n in $R_{\geq n}(\eta)$ is η itself. Therefore the thread returns to η , not to η' .

After returning to η , the same argument applies again: the thread enters $R_{\geq n}(\eta)$ and can only return to η at complexity n . By induction, the thread never visits η' in the eventually $\geq n$ regime, contradicting η' being visited infinitely often. $\square$

2.2 Fragments of $\mathcal{L}_\mu$

Definition 2.11. Guarded Fragment

We call a formula α *guarded* iff every cyclic thread in the subformula graph of α passed through the modal operator $\bigcirc$: for every $\beta \in Sub(\alpha)$ and thread $\tau : \beta \triangleright^* \beta$, if $|\tau| > 1$, then $\tau(i) = \bigcirc\tau(i+1)$ for some $i < |\tau|$.

The second fragment contains all formulas where ν is the only quantifier and μ does not appear. We call this the Π -fragment, and it is given by the following definition.

Definition 2.12. $\Pi(F)$ -fragment

Let $F \subseteq \mathcal{L}_\mu$. The set $\Pi(F)$ is the smallest set of formulas such that:

1. $F \cup Lit \subseteq \Pi(F)$,
2. If $\Gamma \subseteq \Pi(F)$, then $\bigvee \Gamma \in \Pi(F)$ and $\bigwedge \Gamma \in \Pi(F)$,
3. If $\alpha \in \Pi(F)$, then $\bigcirc\alpha \in \Pi(F)$,
4. If $\varphi x \in \Pi(F)$, then $\nu\varphi \in \Pi(F)$,
5. If $\alpha \in \Pi(F)$ is V -positive and $\theta : V \rightarrow \Pi(F)$, then $\alpha[\theta] \in \Pi(F)$.

A $\Pi(\emptyset)$ -formula is called a Π -formula. A Σ -formula is a formula $\alpha^\perp$ for $\alpha \in \Pi(\emptyset)$.

The closure operation $F \rightarrow \Pi(F)$ yields a quantifier hierarchy for $\mathcal{L}_\mu$, which is known as the *alternation depth* or *Niwiński hierarchy*, given as follows: $\Pi_1 = \Pi(\emptyset)$, and $\Pi_{n+1} = \Pi(\Sigma_n)$ where $\Sigma_n = (\Pi_n)^\perp$. Note that this hierarchy exhausts $\mathcal{L}_\mu$.

Lemma 2.13. If $\mu\varphi \in \Pi(F)$, then $\varphi = \psi[\theta]$ for $\mu\psi \in F$ and a substitution $\theta : V \rightarrow \Pi(F)$. The proof of this lemma is given in [1].

2.3 Fixed-point semantics of μ -LTL

The branching nature of modal μ -calculus also informs the authors choice in [1] to evaluate modal μ -calculus formulas against labelled trees. Due to the linearity of μ -LTL, this notion becomes excessive and collapses to linear stream-based semantics. The notion of a tree and the branches it contains collapse because there is always only one branch, so the tree is the branch up to isomorphism. This is why we will replace the tree-based semantics by stream semantics.

We will evaluate μ -LTL formulas against labelled streams, where an L-labelled stream (henceforth simply referred to as *stream*) is a pair $\varsigma = \langle \mathbb{N}, \lambda \rangle$, where $\mathbb{N}$, the set of natural numbers starting at 0, is our time domain and our set of positions. Every stream has the same time domain, which is why we can identify streams simply with the function $\lambda : \mathbb{N} \rightarrow L$, which is the *labelling function* assigning to each time position (or simply position) $i \in \mathbb{N}$ a label $\lambda(i)$. This labelling function returns the set of propositions and variables that are true at the input time position. Writing $x \in \lambda(0)$ means that the variable x holds at time position 0. In contrast to the tree semantics of the modal μ -calculus, the vertices are identified with (time) positions $i \in \mathbb{N}$, and the initial position 0 is called the *root*.

We adopt the usual notion of *immediate successor* of $i = i + 1$ and *immediate predecessor* of $i = i - 1$ of positions as defined for the natural numbers. We write $<$ for the strict successor relation, and $\leq$ for the reflexive transitive closure.

The labelling function of a labelled-path ς is written as λ_ς . For each position $i \in \mathbb{N}$, there exists exactly one suffix $\lambda_\varsigma \uparrow i$ with $\lambda_\varsigma \uparrow i(n) = \lambda_\varsigma(n + i)$ for all $n \in \mathbb{N}$. We denote the immediate suffix λ_{ς_*} as the suffix of the immediate successor.

Definition 2.14. Model

Let $E = 2^{Var}$. A *model* is an E -labelled stream $\lambda : \mathbb{N} \mapsto E$. For our fixed-point semantics, we take $L = E = 2^{Var}$. The set of models is denoted Mod .

Given a model $M = \lambda$ and a set of models $U \subseteq Mod$, observe that for the purpose of evaluating formulas inside M , we only need to consider the suffixes of M in U : $U_M = \{i \in \mathbb{N} \mid M_{\uparrow i} \in U\}$ where $M_{\uparrow i} = \langle \mathbb{N}, \lambda \uparrow i \rangle$. We write $\lambda[x \mapsto U]$ for the function $\lambda' : \mathbb{N} \rightarrow E$ such that $\lambda'(i) = \lambda(i) \cup \{x\}$ if $M_{\uparrow i} \in U$, and $\lambda'(i) = \lambda(i) \setminus \{x\}$ otherwise.

2.3.1 Stream semantics

In stream semantics for μ -LTL each formula α is assigned a set $\|\alpha\| \subseteq Mod$ of models, called the *denotation* of α , and each predicate φ is associated a function $\|\varphi\| : 2^{Mod} \rightarrow 2^{Mod}$, defined by the following mutual recursion:

$$\begin{aligned} \|x\| &= \{\lambda \mid x \in \lambda(0)\} & \|\bigcirc \alpha\| &= \{\lambda \in Mod \mid \lambda_* \subseteq \|\alpha\|\} \\ \|\bigwedge \Gamma\| &= \bigcap \|\Gamma\| & \|\bigvee \Gamma\| &= \bigcup \|\Gamma\| \\ \|\mu\varphi\| &= \bigcap \{U \subseteq Mod \mid \|\varphi\|(U) \subseteq U\} & \|\nu\varphi\| &= \bigcup \{U \subseteq Mod \mid U \subseteq \|\varphi\|(U)\} \end{aligned}$$

where

$$\|\Gamma\| = \{\|\gamma\| \mid \gamma \in \Gamma\} \quad \|\varphi\| : U \mapsto \{\lambda \mid \lambda[x \mapsto U] \in \|\varphi\|\}.$$

If $\|\alpha\| = Mod$, then we call α *valid*.

Proposition 2.15. Suppose λ and λ' are models, and $V \subseteq Var$ is such that for all $i \in \mathbb{N}$, we have $\lambda(i) \cap V = \lambda'(i) \cap V$. Then for every formula α whose free variables are among V , $\lambda \in \|\alpha\|$ iff $\lambda' \in \|\alpha\|$.

Proposition 2.16. The denotation of predicates are monotone functions: If φ is a predicate and $U \subseteq U' \subseteq Mod$, then $\|\varphi\|(U) \subseteq \|\varphi\|(U')$.

Lemma 2.17. $\|\alpha^\perp\| = Mod \setminus \|\alpha\|$ for every formula α .

Proposition 2.18. $\|\varphi\| \|\sigma\varphi\| = \|\sigma\varphi\|$.

2.3.2 Verification semantics

Additionally to the stream semantics, we introduce a Game semantics. Just like in [1], this semantics is formulated as a two-player game. One player aims to *verify* the claim " α is true in λ " while the opponent aims to prevent this. In order to show the validity of α in λ , we require a *verification* in the form of a winning strategy for the verifier.

While in [1], trees are used in this semantic representation due to the branching nature, in linear time we can achieve the same results by considering a linear verification structure which will be called a verification stream.

Definition 2.19. Verification

A *pre-verification* is an $(\mathcal{L}_\mu \times Mod)$ -labelled tree $V = \lambda$ such that for every $i \in \mathbb{N}$:

1. If $\beta = x$: $x \in \lambda(k)$ and the node is a leaf.
2. If $\beta = x^\perp$: $x \notin \lambda(k)$ and the node is a leaf.
3. If $\beta = \bigvee \Gamma$: some child $(\gamma, \lambda|k)$ with $\gamma \in \Gamma$.
4. If $\beta = \bigwedge \Gamma$: for every $\gamma \in \Gamma$, a child $(\gamma, \lambda|k)$.
5. If $\beta = \sigma\varphi$: a child $(\varphi(\sigma\varphi), \lambda|k)$.
6. If $\beta = \bigcirc\gamma$: a child $(\gamma, \lambda|(k+1))$.

A *verification* is a pre-verification in which every infinite branch, viewed as a thread via its formula components, is a ν -thread. We use $\langle |\alpha| \rangle$ to denote the set of models M for which there exists a verification V with root (α, λ) .

Note that predicates can be interpreted as monotone functions over the set of models:

$$\langle |\varphi| \rangle : U \mapsto \{\lambda \mid \lambda[x \mapsto U] \in \langle |\varphi x| \rangle\}.$$

The following proposition translates directly from [1] because the notions in play are structurally unchanged. It shows that the defined verification semantics behave correctly and identify μ as the *least* fixed point and ν as the *greatest* fixed point.

Proposition 2.20. If $\langle |\varphi| \rangle(U) \subseteq U$, then $\langle |\mu\varphi| \rangle \subseteq U$ and if $U \subseteq \langle |\varphi| \rangle(U)$, then $U \subseteq \langle |\nu\varphi| \rangle$.

Combining 2.18 and 2.20, we obtain the equivalence of stream structures and our verification stream semantics, referred to as the *fundamental semantic theorem*. The inductive proof of $|\alpha|$ using the mentioned propositions remains unchanged from [1].

Theorem 2.21. Fundamental Semantic Theorem

For all α , $\|\alpha\| = \langle |\alpha| \rangle$: $\lambda, 0 \models \alpha$ if and only if $\lambda \in \langle |\alpha| \rangle$.

This is standard; see Streett-Emerson [3] and Afshari-Leigh-Menendez Turata [1].

3 Finitary and ill-founded Proof systems

We will deal with *sequents* as the objects that deductions are concerned about. A sequent is an expression of the form $\Phi \Longrightarrow \Psi$ where Φ and Ψ are finite sequences of formulas. We denote the set of all sequents as *Seq.* and the symbols Φ, Ψ , etc. range over this set. In a sequent $\Phi \Longrightarrow \Psi$, we call Φ the *antecedent* and Ψ the *consequent*. We call the sequent *intuitionistic* iff $|\Psi| \leq 1$. When a sequent is intuitionistic, we call the formula in Ψ the *head*, and Φ the context of the sequent. We will also require a clear notion of *occurrence* of formulas in sequents when working in ill-founded proofs, but once this ordering of formulas within sequents will largely be ignored and left implicit.

Definition 3.1. Position A *position* is a pair $p = (s, k) \in \{L, R\} \times \omega$. When $s = L$ we call p a *left* position, otherwise p is a *right* position. k tracks the index of the formula in an ordering in Φ or Ψ . A *formula*

occurrence in a sequent $\mathcal{S} = \Phi \Rightarrow \Psi$ is a position $p = (s, k)$ such that $k < |\Phi|$ if p is a left position, and $k < |\Psi|$ otherwise. We denote the formula at position $p = (s, k)$ in the sequent $\mathcal{S}$ as $\mathcal{S}(p)$ where:

$$(\Phi \Rightarrow \Psi)(p) = \begin{cases} \Phi(k) & \text{if } s = L, \\ \Psi(k) & \text{if } s = R. \end{cases}$$

Definition 3.2. Inference

An *inference* is a pair $(\mathcal{S}, Se)$ where $\mathcal{S}$ is a sequent and Se is a finite set of sequents, usually presented in the form:

$$\frac{Se}{\mathcal{S}}$$

or

$$\frac{\mathcal{S}_0 \cdots \mathcal{S}_n}{\mathcal{S}}$$

When $Se = \{\mathcal{S}_0, \dots, \mathcal{S}_n\}$.

We refer to the sequent $\mathcal{S}$ as the *conclusion* of the inference and the elements of Se as the *premises*. A set of inferences constitutes a *rule*. Typically rules are specified by an inference scheme such as the rule $\bigvee R$ for example:

$$\frac{\Phi \Rightarrow \Psi, \gamma}{\Phi \Rightarrow \Psi, \bigvee \Gamma} \bigvee R$$

for all Φ, Ψ, Γ , and $\gamma \in \Gamma$. An inference is an *instance* of a rule iff it is an element of that rule. For our purposes each rule is further identified as being a *logical* or a *structural* rule. The only difference between these two types of rules is how they are used within derivations and proofs.

Definition 3.3. Derivation

A *derivation* over a set of rules $\mathcal{R}$ is a tree d with labels $\mathcal{S}_d : d \mapsto Seq.$ and $\mathcal{R}_d : d \mapsto \mathcal{R}$ such that:

1. for each vertex $v \in d$, we have that $(\mathcal{S}_d(v), \{\mathcal{S}_d(u) \mid u \in \text{child}(v)\}) \in \mathcal{R}_d(v)$.
2. for each branch $B \subseteq d$, there are infinitely many $i < \omega$ such that $\mathcal{R}_d(B(i))$ is a logical rule.

The *endsequent* of a derivation is the sequent labelling its root. A derivation is *constructive* iff every sequent occurring in the derivation is intuitionistic.

The first condition of the definition ensures that the inferences of the derivation are labelled with proof rules, and that the leaves of the derivation have no premises. The second condition of the definition will become important when we work in an ill-founded proof system which allows certain derivations to have infinite paths. Note that not every derivation is a proof. In finitary proof systems, only finite derivations are allowed as proofs. In the ill-founded proof systems, infinite derivations can be recognised as proofs, but only if their branches fulfil a certain syntactic correctness condition.

3.1 Basic fixed-point logic

We will begin with introducing a system of derivations and proofs that will form the core of all the other proof calculi that will be discussed. We call this system $K_{\bar{\mu}}^{LTL}$ or simply $K_{\bar{\mu}}$. The structural and logical rules of $K_{\bar{\mu}}$ are listed in fig. 1 and 2 below. Rules of this system denoted $\star L$ are *left rules* and $\star R$ are *right rules*. Other rules such as (b) and (id) are both right and left rules and are not further specified. Leaves of a derivation labelled by the (b) rule are called *buds* and allow for conditional proofs, or proofs from assumptions. A finite derivation in $K_{\bar{\mu}}$ is called a $K_{\bar{\mu}}$ -*proof from assumptions* with a $K_{\bar{\mu}}$ -*proof* being a finite derivation without buds.

$$\begin{array}{c}
\frac{\Phi, \alpha, \Lambda \Rightarrow \Psi}{\alpha, \Phi, \Lambda \Rightarrow \Psi} \text{eL} \quad \frac{\Phi \Rightarrow \Psi, \alpha, \Lambda}{\Phi \Rightarrow \Psi, \Lambda, \alpha} \text{eR} \quad \frac{\alpha, \alpha, \Phi \Rightarrow \Psi}{\alpha, \Phi \Rightarrow \Psi} \text{cL} \quad \frac{\Phi \Rightarrow \Psi, \alpha, \alpha}{\Phi \Rightarrow \Psi, \alpha} \text{cR} \\
\\
\frac{\Phi \Rightarrow \Psi}{\alpha, \Phi \Rightarrow \Psi} \text{wL} \quad \frac{\Phi \Rightarrow \Psi}{\Phi \Rightarrow \Psi, \alpha} \text{wR} \quad \frac{}{\Phi \Rightarrow \Psi} \text{b}
\end{array}$$

Figure 1: Structural rules of $K_{\bar{\mu}}$

$$\begin{array}{c}
(\gamma \in \Gamma) \frac{\gamma, \Phi \Rightarrow \Psi}{\bigwedge \Gamma, \Phi \Rightarrow \Psi} \wedge \text{L} \quad \frac{\{\gamma, \Phi \Rightarrow \Psi \mid \gamma \in \Gamma\}}{\bigvee \Gamma, \Phi \Rightarrow \Psi} \vee \text{L} \quad \frac{\varphi(\sigma\varphi), \Phi \Rightarrow \Psi}{\sigma\varphi, \Phi \Rightarrow \Psi} \sigma \text{L} \\
\\
\frac{\{\Phi \Rightarrow \Psi, \gamma \mid \gamma \in \Gamma\}}{\Phi \Rightarrow \Psi, \bigwedge \Gamma} \wedge \text{R} \quad (\gamma \in \Gamma) \frac{\Phi \Rightarrow \Psi, \gamma}{\Phi \Rightarrow \Psi, \bigvee \Gamma} \vee \text{R} \quad \frac{\Phi \Rightarrow \Psi, \varphi(\sigma\varphi)}{\Phi \Rightarrow \Psi, \sigma\varphi} \sigma \text{R} \\
\\
\frac{}{\alpha, \alpha^\perp \Rightarrow \emptyset} \text{idL} \quad \frac{}{\emptyset \Rightarrow \alpha, \alpha^\perp} \text{idR} \quad \frac{}{\alpha \Rightarrow \alpha} \text{id} \\
\\
\frac{\Phi \Rightarrow \Psi}{\bigcirc \Phi \Rightarrow \bigcirc \Psi} \text{next}
\end{array}$$

Figure 2: Logical rules of $K_{\bar{\mu}}$

Proposition 3.4. Any $K_{\bar{\mu}}$ -derivation whose endsequent has an empty consequent is constructive.

Proposition 3.5. There is a $K_{\bar{\mu}}$ -proof of $\Phi \Rightarrow \Psi$ from assumptions $\{\Phi_i \Rightarrow \Psi_i \mid i \in A\}$ iff there is a $K_{\bar{\mu}}$ -proof of $\Psi^\perp, \Phi \Rightarrow \emptyset$ from the assumptions $\{\Psi_i^\perp, \Phi_i \Rightarrow \emptyset \mid i \in A\}$.

Proposition 3.6. The monotonicity rules are derivable in $K_{\bar{\mu}}$. For each φ, α , and β , there is a $K_{\bar{\mu}}$ -proof of $\varphi\alpha \Rightarrow \varphi\beta$ from assumptions $\alpha \Rightarrow \beta$.

3.2 Finitary Proofs

We introduce the first extension of the $K_{\bar{\mu}}$ calculus. We denote $K_{\bar{\mu}}^{LTL}$ or simply K_{μ} as the calculus which extends $K_{\bar{\mu}}$ by initial sequents, the structural rule of cut, and rules of induction. We consider finite derivations with buds and we refer to proofs in K_{μ} as *finitary* proofs. The constructive fragment of K_{μ} is again the restriction of the rules to intuitionistic sequents. This constructive fragment of K_{μ} is denoted cK_{μ}^{LTL} or simply cK_{μ} .

Note that Proposition 3.5 carries over to K_{μ} but Proposition 3.6 does not because of the cut rule.

$$\begin{array}{c}
\frac{\Phi \Rightarrow \Psi, \varphi(\bigwedge(\Phi \bigcup \Psi^\perp))}{\Phi \Rightarrow \Psi, \nu\varphi} \text{indR} \\
\\
\frac{\varphi(\bigwedge(\Phi^\perp \bigcup \Psi))\Phi \Rightarrow \Psi}{\mu\varphi, \Phi \Rightarrow \Psi} \text{indL} \\
\\
\frac{\Phi \Rightarrow \Psi, \alpha \quad \alpha, \Lambda \Rightarrow \Xi}{\Phi, \Lambda \Rightarrow \Psi, \Xi} \text{cut}
\end{array}$$

Figure 3: Induction and Cut rules of K_{μ}

Proposition 3.7. $K_{\mu} \vdash \Phi \Rightarrow \Psi$ iff $K_{\mu} \vdash \Psi^\perp, \Phi \Rightarrow \emptyset$

Proof. By proposition 3.5 it suffices to consider the induction and Cut rules. By replacing the cut rule by $(\vee L)$ with the principal formula $\bigvee(\alpha, \alpha^\perp)$ and then cutting against a K_{μ} -proof of the same formula, we obtain the desired result. $\square$

Later on, we will work with a general formulation of the induction rules, which are referred to as *strong induction* rules.

$$\frac{\Phi \implies \Psi, \nu x \varphi (\bigwedge (\Phi \bigcup \Psi^\perp) \vee x)}{\Phi \implies \Psi, \nu \varphi} \text{SindR}$$

$$\frac{\mu x \varphi (\bigvee (\Phi^\perp \bigcup \Psi) \wedge x), \Phi \implies \Psi}{\mu \varphi, \Phi \implies \Psi} \text{SindL}$$

Figure 4: Strong Induction Rules

Proposition 3.8. The strong induction rules are derivable in K_μ . Instances of strong induction with intuitionistic conclusions are provable in cK_μ .

Theorem 3.9. If $K_\mu \vdash \Phi \implies \Psi$, then $\bigwedge \Phi \rightarrow \bigvee \Psi$ is valid with respect to our stream semantics.

3.3 Ill-founded proofs

We introduce the system $K_{\mu\omega}^{LTL}$ or simply $K_{\mu\omega}$ of *ill-founded proofs* or *co-proofs* for short, which stands for *co-inductive proofs*. $K_{\mu\omega}$ allows for derivations of infinite lengths, and proofs of infinite lengths that express winning strategies in our verification semantics.

Definition 3.10. Let B be a branch of a $K_{\bar{\mu}}$ -derivation d , and let $\mathcal{S}_i$ be the sequent at $B(i)$. A finite trace through B is a sequence of positions $(p_i)_{i \leq k}$ such that p_0 is a position in $\mathcal{S}_0$ and for all $i < k$, we have:

1. if $\mathcal{R}_d(B(i))$ is not structural, then $p_{i+1} = p_i$,
2. if $\mathcal{R}_d(B(i))$ is structural, then p_{i+1} is the formula occurrence corresponding to p_i .

We refer to p_i as the *descendant* of p_0 at $B(i)$. An infinite trace is a sequence such that all finite prefixes are traces. A finite or infinite trace $t = (p_i)_{i < k}$ through B induces a weak thread $w = (\mathcal{S}_i(p_i))_{i < k}$ selecting the formula at position p_i in each $\mathcal{S}_i$. The thread *supported* by t (along B) is the unique thread $\tau = w \circ g$ where $g : L \mapsto K$ is the strictly increasing function such that $j \in g[L]$ iff p_j is principal at $B(j)$. We say that B *carries* τ if τ is an infinite thread that is supported by some trace in B .

We also define for a set of positions P the set $\text{Thd}_P(d, B)$ to be the set of infinite threads supported by a trace t along B such that $t(0) \in P$. If $t(0)$ is a left position, we refer to the thread supported by t as a *left thread* and similarly if $t(0)$ is a right position, we call t a *right thread*. Note that a thread can be both right and left in the same branch. We write $\text{Thd}_P(d, B)$ instead of $\text{Thd}_{\{P\}}(d, B)$, and $\text{Thd}(d, B)$ instead of $\text{Thd}_P(d, B)$ where P is the set of all positions in the root sequent. Given a subsequent $\mathcal{S}$ of the endsequent of d , we may write $\text{Thd}_{\mathcal{S}}(d, B)$ in place of $\text{Thd}_P(d, B)$ where P is the set of positions corresponding to the formulas in $\mathcal{S}$.

Proposition 3.11. Let d be a derivation in $K_{\bar{\mu}}$ with a branch B . If B carries an infinite thread starting in a guarded formula, then every infinite trace through B supports an infinite thread.

Definition 3.12. ill-founded proof

$K_{\mu\omega}$ comprises the rules of $K_{\bar{\mu}}$ with the identity rules restricted to variables. A proof in $K_{\mu\omega}$ is a derivation such that it has no open leaves and every infinite branch carries either a right ν -thread or a left μ -thread. Proofs in $K_{\mu\omega}$ are called *ill-founded proofs* or *co-proofs*.

Remark 3.13. Acceptance condition

Since the right-hand side is always empty, there are no right threads. A branch is *good* if it is closed or if it is infinite and carries at least one μ -thread. A branch is *bad* if it is an open leaf or if it is infinite and carries *no* μ -thread. Crucially, on a bad infinite branch **every** infinite thread is a ν -thread. This is the key property exploited in the truth lemma.

Unlike in K_μ , the cut rule is not permitted in co-proofs. The rule can be added, though the notion of proofhood must be weakend to allow for good threads to initiate from cut formulas. Instead of saying that

every branch carries a thread, we weaken to say that every branch has a suffix which carries a thread. This will become relevant later when we show the reduction of co-proofs to cyclic proofs, and we will show admissability of the cut rule in $K_{\mu\omega}$.

4 Soundness and Completeness of $K_{\mu,\omega}$

4.1 Soundness of $K_{\mu,\omega}$

Theorem 4.1. Soundness of Co-Inductive Proofs

If $K_{\mu\omega} \vdash \Phi \implies \Psi$, then $\Phi \implies \Psi$ is valid.

Theorem 4.2. Soundness of Co-Inductive Proofs (reformulated)

If $K_{\mu}^{\omega} \vdash \alpha \Rightarrow \emptyset$, then α is unsatisfiable.

Proof. Suppose for contradiction that $\lambda, 0 \models \alpha$, witnessed by a verification V of (α, λ) .

Let π be a co-proof of $\alpha \Rightarrow \emptyset$. We construct a branch B of π from V and show that B violates the acceptance condition.

Construction of B

The branch B is determined by V : at each $\bigvee L$ node of π with principal formula $\bigvee \Gamma'$, the verification V selects some disjunct $\gamma_0 \in \Gamma'$ (clause (3) of Definition 2.19); we follow the child of π corresponding to γ_0 . At $\bigwedge L$ nodes, there is a unique child (all conjuncts are placed into the antecedent). At σL and *next* nodes, there is likewise a unique child. Since π has no open leaves (condition (a)), the branch B is infinite.

Every thread along B is a ν -thread

Consider any infinite trace t along B , starting from a formula occurrence in the root sequent. The trace tracks a formula through the rule applications of B , advancing by $\triangleright$ at each rule where the formula is principal. The thread τ supported by t is the sequence of formulas at the principal steps.

We claim τ corresponds to an infinite branch of V . At $\bigvee L$ nodes, t follows the disjunct chosen by V , matching clause (3) of V . At $\bigwedge L$ nodes, t follows one particular conjunct γ ; since V has a child for *every* conjunct (clause (4)), the path through V following γ is present as a branch of V . At σL nodes, t unfolds the fixpoint, matching clause (5). At *next* nodes, t strips the modality, matching clause (6). At literal leaves, t terminates, matching clauses (1)/(2).

Therefore the thread τ supported by t is realized as an infinite branch of V . Since V is a verification, every infinite branch of V is a ν -thread. Hence τ is a ν -thread.

Contradiction

Since every infinite thread carried by B is a ν -thread, the branch B carries no μ -thread, violating condition (b) of Definition 3.12. This completes the proof of soundness. $\square$

4.2 Proof Search

Definition 4.3. Stream depth

We define the notion of depth within a stream as follows:

1. $d(\text{root}) = 0$,
2. $d(v) = d(u)$ if v is obtained from u by $\bigvee L$, $\bigwedge L$, or σL ,
3. $d(v) = d(u) + 1$ if v is obtained by *next*.

Definition 4.4. Modal saturation

$\Gamma \Rightarrow \emptyset$ is *modally saturated* if every $\gamma \in \Gamma$ is a literal or a *next*-formula.

Definition 4.5. Saturated proof search $\text{PS}(\alpha \Rightarrow \emptyset)$

Let α be guarded. The saturated proof search is a tree of sequents, rooted at $\alpha \Rightarrow \emptyset$, built by processing each node $\Gamma \Rightarrow \emptyset$ as follows.

Phase 1 (apply exhaustively until modally saturated):

- 1.1 $\bigwedge L$: if $\bigwedge \Gamma' \in \Gamma$, replace $\bigwedge \Gamma'$ by Γ' .
- 1.2 $\bigvee L$: if $\bigvee \Gamma' \in \Gamma$, create one child branch per $\gamma \in \Gamma'$.
- 1.3 σL : if $\sigma\varphi \in \Gamma$, replace $\sigma\varphi$ by $\varphi(\sigma\varphi)$

Phase 2 (first applicable after modal saturation):

- 2.1 *Identity check*: if Γ contains x and $x^\perp$ for some $x \in \text{Var}$, close by $\text{id}L$.
- 2.2 *Open leaf*: if $\Gamma \subseteq \text{Lit}$ and Γ is consistent, the leaf is open; the branch terminates and it is a bad finite branch in the proof tree.
- 2.3 *next*: if every non-literal in Γ is a *next*-formula, write $\Gamma = L \cup \bigcirc \Gamma'$ with $L \subseteq \text{Lit}$; the child is $\Gamma' \Rightarrow \emptyset$ at depth $d + 1$; return to Phase 1.

Priority: (2.1) $\prec$ (2.2) $\prec$ (2.3).

Remark 4.6. Step (2.1) checks for complementary literals at *every* modally saturated sequent, not only at all-literal leaves. This ensures that on any bad (non-closed) branch, the antecedent is consistent at every modally saturated sequent encountered during the proof search.

Remark 4.7 (Persistence of literals). Once a literal enters the antecedent Γ , it remains present through all subsequent applications of $\bigwedge L$, $\bigvee L$, and σL at the same stream depth: none of these rules remove literals from Γ . The only rule that removes literals is *next*, which drops all literals and advances the stream depth. Therefore the set of literals in Γ grows monotonically within each stream depth, and the final set of literals at depth k (present when *next* fires) includes every literal that appeared at any modally saturated sequent during depth k .

Definition 4.8. Canonical sequent

The *canonical sequent* $S_k(B) = \Gamma_k \Rightarrow \emptyset$ along a branch B at depth k is the modally saturated sequent at depth k produced just before the *next* rule fires (advancing to depth $k+1$). That is, $S_k(B)$ is the *last* modally saturated sequent at depth k : by this point, all fixed-point unfoldings and propositional decompositions at depth k have been performed, and Γ_k contains every literal and *next*-formula accumulated during depth k .

If B terminates at an open or closed leaf at depth k_0 , then $S_{k_0}(B)$ is the modally saturated sequent at which the branch terminates.

Remark 4.9. The canonical sequent is the appropriate choice for defining the stream λ_B :

by Remark 4.7, Γ_k contains every literal encountered at depth k , so any literal reached by the pre-verification at depth k is guaranteed to be in Γ_k . Choosing an earlier modally saturated sequent at depth k would risk missing literals introduced by later σL unfoldings and $\bigwedge L$ decompositions.

Lemma 4.10. Well-definedness For guarded α , the *next* rule fires exactly once at each non-terminal depth along each branch: no branch gets stuck in an infinite loop of Phase 1 and σL applications at a single depth without ever reaching a *next* step. Consequently, $S_k(B)$ is well-defined at every depth k along every non-closed, non-terminated branch.

Proof. Every formula in the proof search belongs to $\text{Sub}(\alpha)$. Suppose *next* never fires from some depth onward along some branch. Then only $\bigvee L$, $\bigwedge L$, and σL apply, generating an infinite thread in $\text{Sub}(\alpha)$ with no *next*-step. Since $\text{Sub}(\alpha)$ is finite, some formula is revisited, producing a cyclic thread. Propositional steps ($\bigvee L$, $\bigwedge L$) strictly decrease complexity and cannot cycle alone, so the cycle must include a fixpoint unfolding through $\sigma\varphi$ —a cyclic thread with no *next*-step, contradicting guardedness (Definition 2.11).

It remains to show that *next* fires *at most* once per depth. After *next* fires at depth k , the stream depth advances to $k + 1$ and the process restarts with Phase 1; no further processing occurs at depth k . $\square$

Definition 4.11 (Branch character). A branch B of $\text{PS}(\alpha \Rightarrow \emptyset)$ is:

- *Closed (good)*: terminated by idL (complementary literals detected).
- *Open (bad finite)*: terminated at a consistent all-literal leaf.
- *Infinite and good*: carries at least one μ -thread.
- *Infinite and bad*: carries *no* μ -thread; every infinite thread is a ν -thread.

A branch is *bad* if it is open or infinite and bad.

Lemma 4.12. If $PS(\alpha \Rightarrow \emptyset)$ has no bad branches, then $K_\mu^\omega \vdash \alpha \Rightarrow \emptyset$.

Proof. Every branch is good: closed or infinite with at least one μ -thread. The proof search tree is a co-proof. $\square$

Lemma 4.13. If $K_\mu^\omega \not\vdash \alpha \Rightarrow \emptyset$, then $PS(\alpha \Rightarrow \emptyset)$ has at least one bad branch.

Proof. Contrapositive of Lemma 4.12. $\square$

Remark 4.14 (Extraction of bad branches and the role of Dependent Choice). The proof search $PS(\alpha \Rightarrow \emptyset)$ is a finitely branching tree: at each $\bigvee L$ node, the number of children equals $|\Gamma'|$, which is finite; all other rules produce a single child. Lemma 4.12 states that if every branch is good, the tree is a co-proof. Its contrapositive (Lemma 4.13) asserts the existence of a bad branch when the tree is not a co-proof.

For *finite* bad branches (open leaves), their existence is immediate: the open leaf is itself the witness. For *infinite* bad branches, extracting one from a finitely branching tree requires König's Lemma (equivalently, Dependent Choice): if $PS(\alpha \Rightarrow \emptyset)$ is infinite and not every branch is good, then some infinite path through the tree carries no μ -thread. More precisely, we select a path through the tree by choosing, at each $\bigvee L$ node, a child whose subtree is not a co-proof (such a child exists since at least one subtree must fail to be a co-proof). This iterative selection produces an infinite branch B , and the failure of the co-proof condition along each finite prefix of B ensures that B carries no μ -thread.

4.3 Canonical Verification Stream

Definition 4.15. Canonical stream

Let B be a bad branch of $PS(\alpha \Rightarrow \emptyset)$. Define $\lambda_B: \mathbb{N} \rightarrow \mathcal{P}(Var)$ by

$$\lambda_B(k) = \{x \in Var \mid x \in \Gamma_k\},$$

where Γ_k is the antecedent in the canonical sequent $S_k(B) = \Gamma_k \Rightarrow \emptyset$. For a bad finite branch terminating at depth k_0 , set $\lambda_B(k) = \emptyset$ for $k > k_0$.

Remark 4.16. Since B is bad (hence not closed), step (2.1) of the proof search never fired along B : the set Γ_k is consistent at every depth k . Therefore $x \in \lambda_B(k)$ implies $x^\perp \notin \Gamma_k$, and $x^\perp \in \Gamma_k$ implies $x \notin \lambda_B(k)$.

4.4 Truth Lemma

The truth lemma is the core of the completeness argument. Given a bad branch B , we construct a pre-verification of (α, λ_B) by mirroring the proof search along B .

At conjunction nodes the pre-verification branches into all conjuncts; this is sound because the $\bigwedge L$ rule places all conjuncts into the antecedent, so each conjunct is tracked along B . Since the branch is bad, every infinite thread it carries is a ν -thread, which makes the verification condition hold for the entire pre-verification tree including branches through different conjuncts.

Lemma 4.17. Truth Lemma

Let α be guarded and let B be a bad branch of $PS(\alpha \Rightarrow \emptyset)$. Then $\lambda_B, 0 \models \alpha$.

Proof. We construct a pre-verification V_B of (α, λ_B) , show it is a verification, and conclude $\lambda_B \in \langle |\alpha| \rangle$.

Construction of V_B V_B is a tree labelled by pairs $(\beta, \lambda_B \upharpoonright k)$, where β is a formula occurrence in the antecedent at some node of B at depth k , and $\lambda_B \upharpoonright k$ is the shifted stream. The root is (α, λ_B) . The children of each node $(\beta, \lambda_B \upharpoonright k)$ are determined by the proof search rule applied to β along B . More precisely, we track β as a *formula occurrence* through the rule applications of B : at each rule application, β either is the principal formula (and is replaced by its immediate subformula(s)) or is a side formula (and persists unchanged). The children of the node in V_B correspond to the immediate subformula(s) when β is principal:

- $\beta = x$ (**positive literal, leaf**): At depth k , the literal x is present in the antecedent and, by Remark 4.7, persists until *next* fires. Therefore $x \in \Gamma_k$, giving $x \in \lambda_B(k)$ by Definition 4.15, which satisfies clause (1).
- $\beta = x^\perp$ (**negative literal, leaf**): Since Γ_k is consistent (Remark 4.16), $x \notin \Gamma_k$, so $x \notin \lambda_B(k)$, satisfying clause (2).
- $\beta = \bigvee \Gamma' (\bigvee L)$: The proof search along B selects some $\gamma_0 \in \Gamma'$. The unique child is $(\gamma_0, \lambda_B \upharpoonright k)$, satisfying clause (3).
- $\beta = \bigwedge \Gamma' (\bigwedge L)$: The proof search places *all* conjuncts $\gamma \in \Gamma'$ into the antecedent. For *each* $\gamma \in \Gamma'$, we create a child $(\gamma, \lambda_B \upharpoonright k)$, satisfying clause (4). Each child's sub-tree is then constructed by following γ 's formula occurrence through the subsequent rule applications along B . This is well-defined because after $\bigwedge L$, each γ is a formula occurrence in the antecedent and can be tracked independently.
- $\beta = \sigma\varphi (\sigma L)$: The child is $(\varphi(\sigma\varphi), \lambda_B \upharpoonright k)$, satisfying clause (5).
- $\beta = \bigcirc\gamma$ (*next*): The child is $(\gamma, \lambda_B \upharpoonright (k+1))$, satisfying clause (6).

V_B is a pre-verification

Each case satisfies the corresponding clause of Definition 2.19, and the root is (α, λ_B) .

Every infinite branch of V_B corresponds to a trace along B

Consider an infinite branch b of V_B . By construction, b is a sequence of nodes $(\beta_0, \lambda_B \upharpoonright k_0), (\beta_1, \lambda_B \upharpoonright k_1), \dots$ where each β_i is a formula occurrence in the antecedent at the corresponding node of B , and $\beta_i \triangleright \beta_{i+1}$ for each i . The sequence $(\beta_i)_i$ is therefore an infinite thread in $Sub(\alpha)$.

We claim this thread is *carried by* B . Formally, the sequence of formula occurrences tracked through B constitutes a *trace* through B : at each rule application along B , the formula occurrence either is principal (advancing $\beta_i \triangleright \beta_{i+1}$) or is a side formula (persisting to the next sequent without contributing a step to the thread). At $\bigwedge L$ nodes, the branch b selects one conjunct, whose formula occurrence then continues to be tracked in the antecedent. At $\bigvee L$ nodes, b follows the disjunct chosen by B . In all cases, the formula occurrence remains in the antecedent of the sequents along B and the resulting infinite thread is carried by B .

V_B is a verification

Since B is bad, it carries *no* μ -thread (Definition 4.11). By Proposition 2.8, every infinite thread is either μ or ν . Therefore every infinite thread carried by B is a ν -thread. Since every infinite branch b of V_B induces a thread carried by B (as shown above), every infinite branch of V_B is a ν -thread. Therefore V_B is a verification.

Conclusion

We consider the two cases of bad branches.

Case 1: B is bad finite (open leaf). The branch terminates at a consistent set of literals. Every formula occurrence tracked in V_B eventually reaches a literal and terminates, so V_B is finite. The verification condition holds vacuously.

Case 2: B is bad infinite. V_B may be infinite, with potentially many infinite branches (one per choice of conjunct at each $\bigwedge$ node along b). By the argument above, every such branch is a ν -thread, so V_B is a verification.

In both cases V_B is a verification of (α, λ_B) , whence $\lambda_B \in \langle |\alpha| \rangle$. By Theorem 2.21, $\lambda_B, 0 \models \alpha$. $\square$

Remark 4.18. The argument at $\bigwedge$ nodes is the crux of the proof. Two observations make it work:

1. The $\bigwedge L$ rule places *all* conjuncts into the antecedent of the proof search, so every conjunct is tracked along B and can serve as the root of a sub-pre-verification.
2. The acceptance condition requires a good branch to carry at least one μ -thread. Equivalently, a bad branch carries *no* μ -thread: *all* threads are ν -threads. This ensures the verification condition for *every* branch of V_B , not just those following a single designated thread.

Remark 4.19. Role of the canonical sequent

The choice of the canonical sequent as the *last* modally saturated sequent at each depth (Definition 4.8) is essential for the literal cases of the pre-verification.

A literal x may enter the antecedent at depth k only after several σL unfoldings and $\bigwedge L$ decompositions. For instance, if x is buried inside a fixpoint formula that is unfolded at depth k . The canonical sequent, taken just before *next* fires, captures *all* literals accumulated during depth k (Remark 4.7), ensuring that the stream $\lambda_B(k)$ records every literal that the pre-verification may encounter at this depth.

4.5 Completeness of $K_{\mu, \omega}$

Theorem 4.20. Completeness for the guarded fragment

Let α be guarded. If α is unsatisfiable, then $K_\mu^\omega \vdash \alpha \Rightarrow \emptyset$.

Proof. By contrapositive. Suppose $K_\mu^\omega \not\vdash \alpha \Rightarrow \emptyset$. By Lemma 4.13, $\text{PS}(\alpha \Rightarrow \emptyset)$ has a bad branch B . By the Truth Lemma (Lemma 4.17), $\lambda_B, 0 \models \alpha$. Therefore α is satisfiable. $\square$

Corollary 4.21. For every guarded formula α ,

$$K_\mu^\omega \vdash \alpha \Rightarrow \emptyset \iff \alpha \text{ is unsatisfiable.}$$

Proof. Soundness is Theorem 4.2; completeness is Theorem 4.20. $\square$

Remark 4.22 (Summary). The argument has four components:

1. *Proof search* (Section 4.2): For guarded α , the saturated proof search generates a co-proof if and only if it has no bad branches. The identity rule is checked at every modally saturated sequent, ensuring consistency of the antecedent along non-closed branches. The canonical sequent at each depth is taken just before *next* fires, capturing all accumulated literals. Extracting an infinite bad branch from a non-proof requires König's Lemma (Remark 4.14).
2. *Canonical stream* (Section 4.3): From a bad branch B , a stream λ_B is extracted from the positive literals in the canonical sequent at each depth.
3. *Truth lemma* (Section 4.4): A pre-verification of (α, λ_B) is built from B by tracking formula occurrences through rule applications. At conjunction nodes it branches into all conjuncts (each tracked along B by $\bigwedge L$). Each infinite branch of the pre-verification induces a trace along B supporting an infinite thread. Since B is bad, every such thread is a ν -thread, so the pre-verification is a verification.
4. *Completeness* (Section 4.5): Unprovability implies a bad branch, which implies a satisfying stream.

References

- [1] Bahareh Afshari, Graham Leigh, and Guillermo Turata. Demystifying μ . *Fundamenta Informaticae*, Volume 194, Issue 2: Fixed..., 08 2025. doi:[10.46298/fi.12773](https://doi.org/10.46298/fi.12773).
- [2] Christian Dax, Martin Hofmann, and Martin Lange. A proof system for the linear time μ -calculus. pages 273–284, 2006. doi:[10.1007/11944836_26](https://doi.org/10.1007/11944836_26).
- [3] Robert S. Streett and E. Allen Emerson. An automata theoretic decision procedure for the propositional mu-calculus. *Information and Computation*, 81(3):249–264, 1989. URL: <https://www.sciencedirect.com/science/article/pii/089054018990031X>, doi:[10.1016/0890-5401\(89\)90031-X](https://doi.org/10.1016/0890-5401(89)90031-X).